

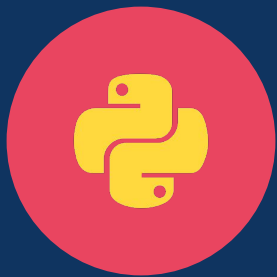
nanoGPT

架构学习



从 Python 基础到 Transformer 全链路

Python · 线性代数 · 注意力机制 · 预训练 · LoRA



模块 一

Python 语法基础

nanoGPT 代码中的必备语法

基础数据结构



list

有序可变序列

```
[1, 2, 3]
```

dict

键值对映射

```
{'a': 1}
```

tuple

有序不可变序列

```
(1, 2, 3)
```

set

无序去重集合

```
{1, 2, 3}
```

nanoGPT 实例: 从文本构建字符表

```
text = "hello"
```

```
set(text)          → {'h', 'e', 'l', 'o'}    # 去重
```

```
list(set(text))    → ['h', 'e', 'l', 'o']    # 转 list
```

```
sorted(...)        → ['e', 'h', 'l', 'o']    # 排序
```

列表推导式与 enumerate



推导式写法

```
chars = ['e','h','l','o']  
  
stoi = {ch: i  
        for i, ch  
        in enumerate(chars)}
```

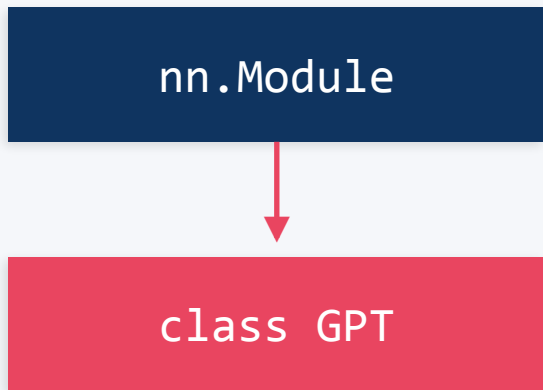
等价 for 循环

```
stoi = {}  
for i, ch in enumerate(chars):  
    stoi[ch] = i
```

输出结果

i	ch	stoi[ch]=i
0	'e'	{'e': 0}
1	'h'	{'e':0, 'h':1}
2	'l'	{'e':0, 'h':1, 'l':2}
3	'o'	{'e':0, 'h':1, 'l':2, 'o':3}

类与继承(nn.Module)



__init__
forward
self

初始化模型组件(层、参数)
定义数据如何流过模型
引用实例本身, 存储属性

nanoGPT 模型骨架

```
class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.transformer = ...
        self.lm_head = ...

    def forward(self, idx):
        # 前向传播逻辑
        return logits, loss

    def generate(self, idx):
        # 自回归生成
```

装饰器与上下文管理器



训练模式 Training

```
# 梯度开启(默认)
model.train()
loss = model(x, y)
loss.backward() # 计算梯度
optimizer.step() # 更新参数
```

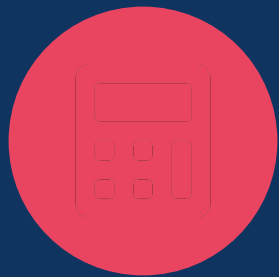
✓ 梯度计算 ✓ Dropout 激活 ✓ 参数更新

推理模式 Inference

```
@torch.no_grad()
def generate(self, idx):
    model.eval()
    with torch.autocast():
        logits = model(idx)
```

✗ 不计算梯度 ✗ Dropout 关闭 → 节省显存

@torch.no_grad() 是装饰器——在函数外面包一层，自动关闭梯度。with 是上下文管理器——在代码块内自动管理资源。

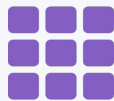


模块二

数学基础

Transformer 背后的核心数学概念

向量与矩阵



向量 Vector

一个 token 的 embedding

0.2
-0.1
0.8

维度 = 3

矩阵 Matrix

一个 batch 的 embeddings

token ₁	0.2	-0.1	0.8
token ₂	0.5	0.3	-0.2
token ₃	-0.4	0.7	0.1

(3×3) 矩阵 = 3 个向量堆叠

Transformer 中, token 用向量表示, 一个句子/ batch 用矩阵表示。所有计算本质上都是矩阵运算。

向量内积(点积)

$$a \cdot b = \sum(a_i \times b_i) \quad \text{对应元素相乘, 再求和} \rightarrow \text{得到一个标量}$$

完整计算示例

a =

1

2

3

b =

0.5

-0.3

0.8

$$\begin{aligned} a \cdot b &= (1 \times 0.5) + (2 \times (-0.3)) + (3 \times 0.8) \\ &= 0.5 + (-0.6) + 2.4 \\ &= 2.3 \end{aligned}$$

在 Attention 中:

Query · Key = 相似度
点积越大 → 越相关

矩阵转置



行列互换: $(m \times n) \rightarrow (n \times m)$

A (2×3)

1	2	3
4	5	6

→ 转置 →

A^T (3×2)

1	4
2	5
3	6

元素位置变化: $A[i][j] \rightarrow A^T[j][i]$

在 nanoGPT 中的用途:

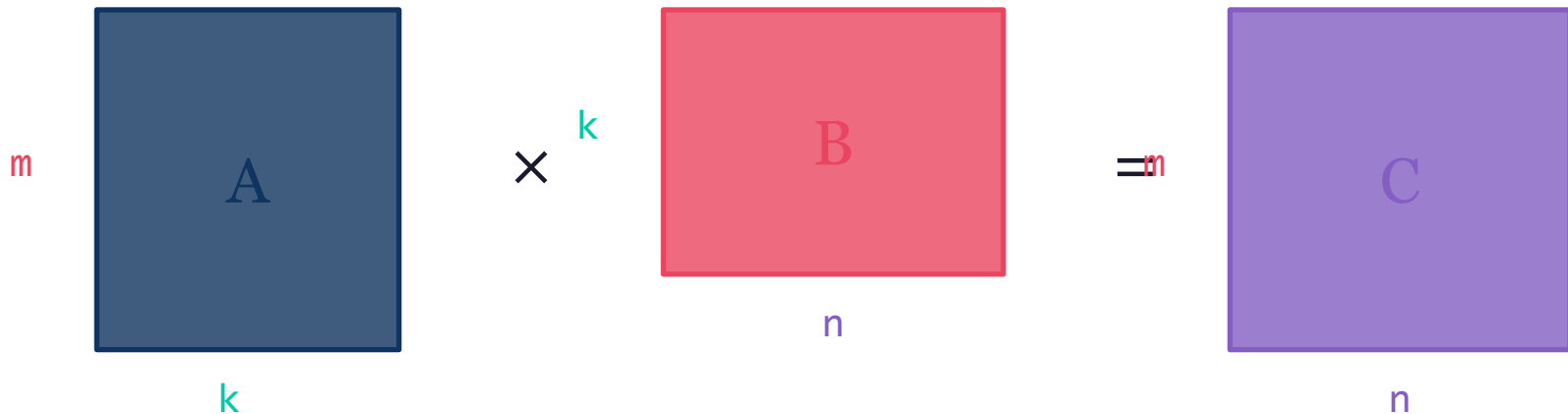
`Q @ K.transpose(-2, -1)`

K 转置后, $Q(T \times d) \times K^T(d \times T) = (T \times T)$ 注意力得分矩阵

矩阵乘法(一) 规则

$$(m \times k) \times (k \times n) \rightarrow (m \times n)$$

中间维度 k 必须相同



$C_{ij} = A \text{ 的第 } i \text{ 行} \cdot B \text{ 的第 } j \text{ 列 (内积)}$

矩阵乘法(二)完整推 导

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

C₁₁ A 行1 [1,2] · B 列1 [5,7]

$$(1 \times 5) + (2 \times 7) = 5 + 14 = \mathbf{19}$$

C₁₂ A 行1 [1,2] · B 列2 [6,8]

$$(1 \times 6) + (2 \times 8) = 6 + 16 = \mathbf{22}$$

C₂₁ A 行2 [3,4] · B 列1 [5,7]

$$(3 \times 5) + (4 \times 7) = 15 + 28 = \mathbf{43}$$

C₂₂ A 行2 [3,4] · B 列2 [6,8]

$$(3 \times 6) + (4 \times 8) = 18 + 32 = \mathbf{50}$$

Softmax

$$\text{softmax}(z_i) = e^{z_i} / \sum e^{z_j}$$

任意实数 $\rightarrow$ 概率分布 (> 0 , 求和 = 1)

完整推导

输入: $z = [2.0, 1.0, 0.1]$

Step 1 指数化:

$$e^{2.0} = 7.39, e^{1.0} = 2.72, e^{0.1} = 1.11$$

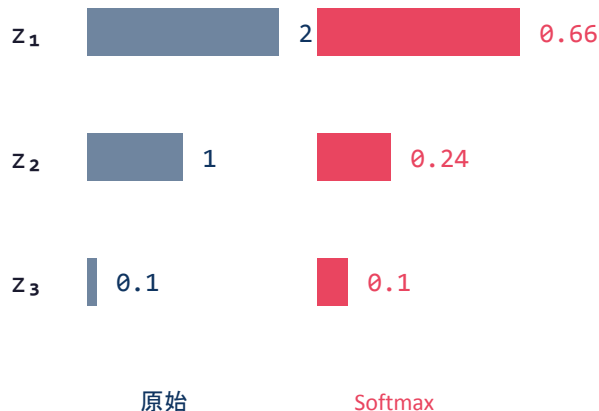
Step 2 求和:

$$7.39 + 2.72 + 1.11 = 11.22$$

Step 3 归一化:

$$[7.39/11.22, 2.72/11.22, 1.11/11.22] \\ = [0.66, 0.24, 0.10]$$

原始分数 vs 概率



交叉熵 Cross Entropy

$$L = -\sum y_i \log(p_i) \quad \text{one-hot 简化为} \quad L = -\log(p_{\text{correct}})$$

预测一般

真实: [0, 0, 1, 0]

预测: [0.1, 0.2, 0.6, 0.1]

$$\text{Loss} = -\ln(0.6) = 0.51$$

预测变好

真实: [0, 0, 1, 0]

预测: [0.05, 0.05, 0.85, 0.05]

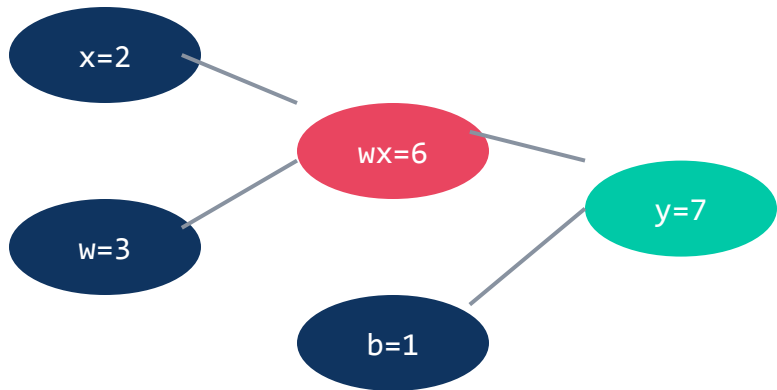
$$\text{Loss} = -\ln(0.85) = 0.16 \quad \downarrow \text{更低!}$$

p_{correct} 越接近 1 $\rightarrow -\ln(p)$ 越接近 0 $\rightarrow$ Loss 越小 $\rightarrow$ 模型越好

梯度与反向传播



计算图: $y = wx + b$



反向传播(链式法则)

假设 $L = (y - \text{target})^2$

$$\partial L / \partial y = 2(y - \text{target})$$

$$\begin{aligned} \partial L / \partial w &= \partial L / \partial y \times \partial y / \partial w \\ &= \partial L / \partial y \times x \end{aligned}$$

$$\begin{aligned} \partial L / \partial b &= \partial L / \partial y \times \partial y / \partial b \\ &= \partial L / \partial y \times 1 \end{aligned}$$

PyTorch 一行搞定:

loss.backward()

核心循环: forward $\rightarrow$ 算 loss $\rightarrow$ backward(算梯度) $\rightarrow$ optimizer.step(更新参数) $\rightarrow$ 重复



模块 三

nanoGPT 核心知识点

从 Tokenizer 到 Transformer 完整拆解

B, T, C

贯穿 nanoGPT 代码的三个核心维度约定

B

Batch Size
= 并发量

一次喂给模型多少条独立文本序列

B=4 → 同时处理 4 段文本

nanoGPT: batch_size

类比: 餐厅同时接待 4 桌客人

T

Sequence Length
= 上下文窗口 长度

每条序列包含多少个 token

T=8 → 每条序列看 8 个 token

nanoGPT: block_size

类比: 每篇文章限读 8 个字

C

Embedding Dim
= 特征/属性 维度

每个 token 用多少维向量表示

C=64 → 每个 token 是 64 维向量

nanoGPT: n_embd

类比: 用 64 种属性描述一个字

数据流维度变化

输入 Token IDs

(B, T)

整数矩阵

Embedding

(B, T, C)

ID → C 维向量

Transformer

(B, T, C)

信息提炼

Output Head

(B, T, V)

预测分布

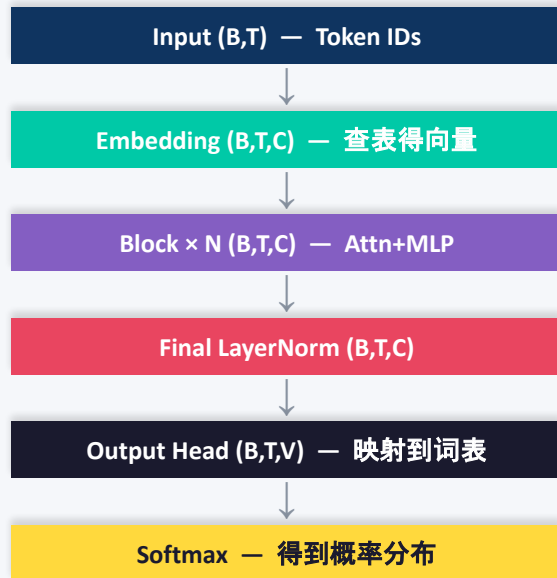
核心直觉

B 影响训练速度(并行度), 不影响模型 结构

T 决定上下文窗口大小 | C 决定模型表达能力 | 全流程: (B,T) → (B,T,C) → (B,T,V)

Transformer 完整数据流 总览

从输入 Token 到输出概率 — 用「天气真好」完整走一遍



① 输入:「天气真好」→ Token IDs

Tokenizer 分词: [天, 气, 真, 好] → [1024, 2048, 512, 768] shape: (1,4)

② Embedding: ID → 向量

$\text{tok_emb}(1024) + \text{pos_emb}(0) = x_0$ shape: (1,4,384) C=384

③ Block × 6: 注意力 + MLP

每个 Block: $x = x + \text{Attn}(\text{LN}(x))$, $x = x + \text{MLP}(\text{LN}(x))$ 保持 (1,4,384)

④ Final LN → Output Head → Softmax

$\text{LN}(x) \rightarrow (1,4,384) @ W(384,V) \rightarrow \text{logits}(1,4,V) \rightarrow \text{softmax} \rightarrow P(\text{下一词})$

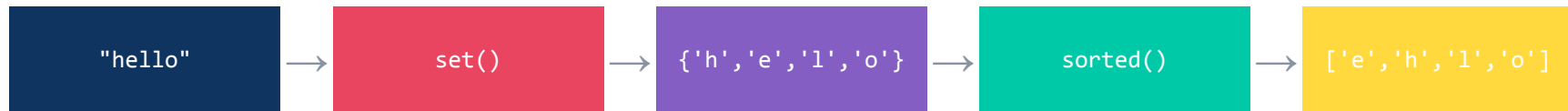
预测结果(位置 3,「好」之后)

$\text{logits}[3] \rightarrow \text{softmax} \rightarrow P(\text{"啊"})=0.35, P(\text{"呀"})=0.22, P(\text{"!"})=0.18 \dots$

模型从 4 个 token 的上下文中综合判断, 生成下一个 词的概率分布

✨ 核心洞察: 整个流程的维度变化 $(B,T) \rightarrow (B,T,C) \rightarrow \dots \rightarrow (B,T,V) \rightarrow \text{概率分布}$, 只有 Embedding 和 Output Head 改变维度

字典构建 (Tokenizer)



双向映射

char → int (stoi)	
'e'	0
'h'	1
'l'	2
'o'	3

编码 "hello"

'h' → 1, 'e' → 0, 'l' → 2, 'l' → 2, 'o' → 3

"hello" → [1, 0, 2, 2, 3]

BPE(子词分词): 实际 GPT 使用 Byte-Pair Encoding, 将高频字符对合并为子词 token。例如 "th" + "e" → "the" 作为一个 token。nanoGPT 教学版用字符级分词简化管理。

BPE 字节对编码

Byte-Pair Encoding — GPT 系列的实际 Tokenizer 方案

BPE 合并过程示例

初始词表: [a, b, c, d, e, r]

语料: a b r a c a d a b r a

第 1 轮 最频繁 pair = (a, b) → 合并为 "ab"

ab r a c a d ab r a

第 2 轮 最频繁 pair = (ab, r) → 合并为 "abr"

abr a c a d abr a

第 3 轮 最频繁 pair = (abr, a) → 合并为 "abra"

abra c a d abra

第 4 轮 最频繁 pair = (a, d) → 合并为 "ad"

abra c ad abra

最终词表: [a, b, c, d, e, r, ab, abr, abra, ad]

词表从 6 → 10, 高频组合获得独立 token

Byte-level BPE (GPT 实际方案)

基础词表 = 256 个 UTF-8 字节

在此基础上做 BPE 合并

优势:

- 任何语言都能处理(中/日/韩/emoji)
- 不会出现 unknown token
- GPT-2 词表大小: 50,257

三种分词方式对比

字符级	nanoGPT 教学版	'h','e','l','l', 'o'
子词级 (BPE)	GPT-2/3/4 实际	'he1','lo'
字节级 BPE	GPT 最终方案	UTF-8字节→合并

与 nanoGPT 的关系

nanoGPT 教学版用字符级分词简化管理 · 实际 GPT-2/3/4 使用 Byte-level BPE · 原理相同, 粒度不同

Embedding (词嵌入)



`nn.Embedding(vocab_size, n_embd)` – 本质是一个查找表

Embedding 查找表 (vocab=4, dim=3)

ID 0 ('e')	0.12	-0.34	0.56
ID 1 ('h')	0.78	0.23	-0.11
ID 2 ('l')	-0.45	0.67	0.33
ID 3 ('o')	0.91	-0.08	0.44

← token ID=1 查表

得到 [0.78, 0.23, -0.11]

为什么需要 Embedding?

整数 ID 没有语义信息

向量可以表达相似性

"猫"和"狗"的向量距离近

位置编码 Positional Encoding

Transformer 没有内置的顺序感知 → 需要位置编码告诉模型 "这是第几个 token"

Token Embedding + Position Embedding = 最终输入

Token Emb	0.78	0.23	-0.11	+			
Position Emb	0.10	-0.05	0.20	=	最终输入		
					0.88	0.18	0.09

```
self.wpe = nn.Embedding(block_size, n_embd) # 可学习的位置编码
```

输入构建(X 和 Y)



自回归训练: 给定前面的 token, 预测下一个 token

文本: "abcdef" block_size=4

X = a b c d Y = b c d e

训练时逐步扩展上下文:

输入 [a] → 预测 b

输入 [a, b] → 预测 c

输入 [a, b, c] → 预测 d

输入 [a, b, c, d] → 预测 e

QKV 构建



$(T \times d)$

Q

Query

$X \times W_q$

我在找什么？

K

Key

$X \times W_k$

我有什么标签？

V

Value

$X \times W_v$

我的信息内容

图书馆类比

Q = 你的搜索词 "我想找关于机器学习的书"

K = 每本书的标签 "AI""深度学习""烹饪""历史"...

V = 书的内容 Q-K 匹配度高的书 → 取出对应 V(内容)

自注意力机制(一)公式

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \times V$$

Step 1

$$Q \cdot K^T$$

算出每对 token 的相关性得分
维度: $(T \times d) \times (d \times T) = (T \times T)$

Step 2

$$\div \sqrt{d}$$

缩放防止数值过大
避免 softmax 梯度消失

Step 3

Softmax

归一化为概率分布
每行和 = 1

Step 4

$$\times V$$

用概率加权汇总信息
 $(T \times T) \times (T \times d) = (T \times d)$

维度流: $Q(T \times d) \times K^T(d \times T) \rightarrow \text{得分}(T \times T) \rightarrow \text{softmax}(T \times T) \rightarrow \times V(T \times d) \rightarrow \text{输出}(T \times d)$

自注意力机制(二)数值示例

3 个 token, d=2 的小例子

Q	1	0	K	1	1	V	0.5	0.3	$Q \cdot K^T / \sqrt{2} =$	0.71	0.00	0.71
	0	1		0	1		0.2	0.8		0.71	0.71	0.00
	1	1		1	0		0.9	0.1		1.41	0.71	0.71

Softmax 后的注意力 权重(热力图)

	t_1	t_2	t_3
t_1	0.39	0.22	0.39
t_2	0.36	0.36	0.28
t_3	0.40	0.30	0.30

最终输出 = Weights $\times$ V

$$\begin{aligned} t_1: & 0.39 \times [0.5, 0.3] + 0.22 \times [0.2, 0.8] \\ & + 0.39 \times [0.9, 0.1] \\ & = [0.59, 0.33] \end{aligned}$$

每个 token 的输出 = 对所有 V 的
加权平均, 权重由 Q·K 决定

具体例子 — 用数字走一遍 BTC

B=2, T=4, C=3, V=5 · 词表=['天','气','真','好','冷']

输入矩阵 (2, 4)

序列0: "天气真好" → [0, 1, 2, 3]

序列1: "天气真冷" → [0, 1, 2, 4]

每个 token 只是一个编号, 没有任何语义信息

nn.Embedding 权重矩阵 (5×3)

权重 W — 用 token ID 当行号查表

ID 0 '天' → [0.8, -0.2, 0.5]

ID 1 '气' → [-0.1, 0.9, 0.3]

ID 2 '真' → [0.4, 0.4, -0.6]

ID 3 '好' → [0.7, 0.1, 0.8]

ID 4 '冷' → [-0.5, 0.6, -0.3]

Embedding 后 (2,4,3) — 每个编号变成 3 维语义向量

序列 0 "天气真好"

天: [0.9, -0.2, 0.4]

气: [-0.1, 1.0, 0.3]

真: [0.3, 0.4, -0.5]

好: [0.7, 0.0, 0.8]

序列 1 "天气真冷"

天: [0.9, -0.2, 0.4] ← 相同

气: [-0.1, 1.0, 0.3] ← 相同

真: [0.3, 0.4, -0.5] ← 相同

冷: [-0.5, 0.5, -0.3] ← 不同!

关键观察

两条序列前 3 个 token 完全相同, 只有最后一个不同(好vs 冷)——这将导致 Transformer 输出不同的预测

Embedding 权重 = “语义身份证”

`nn.Embedding(V, C)` 内部的权重矩阵 W , 形状 $(V \times C)$

Token Embedding 查找表

```
self.wte = nn.Embedding(5, 3)
```

	维度0	维度1	维度2	
ID 0	天	[0.8, -0.2, 0.5]	←	第0行
ID 1	气	[-0.1, 0.9, 0.3]	←	第1行
ID 2	真	[0.4, 0.4, -0.6]	←	第2行
ID 3	好	[0.7, 0.1, 0.8]	←	第3行
ID 4	冷	[-0.5, 0.6, -0.3]	←	第4行

查表: 输入 ID=2 → 取第2行 → [0.4, 0.4, -0.6]

Position Embedding ($T \times C$)

```
self.wpe = nn.Embedding(T, 3)
```

位置 0	→	[0.1, 0.0, -0.1]
位置 1	→	[0.0, 0.1, 0.0]
位置 2	→	[-0.1, 0.0, 0.1]
位置 3	→	[0.0, -0.1, 0.0]

关键点

- 初始值随机, 训练后自动学出语义
- 语义相近的词会获得相近的向量
- 占总参数量 $V \times C$
- 最终输入 = `tok_emb + pos_emb`

`tok_emb + pos_emb = 最终输入向量(序列 0)`

天: [0.8, -0.2, 0.5] + [0.1, 0.0, -0.1] = [0.9, -0.2, 0.4]
真: [0.4, 0.4, -0.6] + [-0.1, 0.0, 0.1] = [0.3, 0.4, -0.5]

气: [-0.1, 0.9, 0.3] + [0.0, 0.1, 0.0] = [-0.1, 1.0, 0.3]
好: [0.7, 0.1, 0.8] + [0.0, -0.1, 0.0] = [0.7, 0.0, 0.8]

本质

所谓“查表”就是用 token ID 当行号取出对应行——这个矩阵就是模型参数的一部分, 训练时通过反向传播不断调整

Self-Attention 详解 — Q/K/V 生成

以序列 o 位置 2 的“真”为例, 看它如何融入上下文

Step 1: 生成 Q、K、V

三个权重矩阵 W_q 、 W_k 、 W_v (3×3):

$W_q \rightarrow$ “提问者” $W_k \rightarrow$ “标签” $W_v \rightarrow$ “内容”

$$Q_2 = x_2 \times W_q = [0.3, 0.4, -0.5] \quad \leftarrow \text{真在找什么}$$

$$K_0 = x_0 \times W_k = [-0.2, 0.9, 0.4] \quad \leftarrow \text{天的标签}$$

$$K_1 = x_1 \times W_k = [1.0, -0.1, 0.3] \quad \leftarrow \text{气的标签}$$

$$K_2 = x_2 \times W_k = [0.4, 0.3, -0.5] \quad \leftarrow \text{真的标签}$$

Step 2: 算注意力分数 ($Q \cdot K^T$)

$$\begin{aligned} \text{score}(\text{真} \rightarrow \text{天}) &= Q_2 \cdot K_0 \\ &= 0.3 \times (-0.2) + 0.4 \times 0.9 + (-0.5) \times 0.4 \\ &= -0.06 + 0.36 - 0.20 = 0.10 \end{aligned}$$

$$\text{score}(\text{真} \rightarrow \text{气}) = 0.11$$

$$\text{score}(\text{真} \rightarrow \text{真}) = 0.49 \quad \leftarrow \text{最高}$$

$$\text{score}(\text{真} \rightarrow \text{好}) = -\infty \quad (\text{因果遮罩!})$$

直觉

Q·K 点积越大 = 两个 token 越“相关”——因果遮罩禁止看到未来的“好”

Value 向量(被选中时贡献的信息)

$$V_0(\text{天}) = x_0 \times W_v = [1.3, -0.2, 0.4]$$

$$V_1(\text{气}) = x_1 \times W_v = [0.2, 1.0, 0.3]$$

$$V_2(\text{真}) = x_2 \times W_v = [-0.2, 0.4, -0.5]$$

Self-Attention — Softmax 与加权求和

把注意力分数转为权重, 然后加权混合 Value 向量

Step 3: 缩放 + Softmax → 注意力权重

除以 $\sqrt{d} = \sqrt{3} \approx 1.73$:

$[0.10, 0.11, 0.49] / 1.73 = [0.058, 0.064, 0.283]$

Softmax →

天: 0.307 (30.7%) ← 吸收天的信息
气: 0.309 (30.9%) ← 吸收气的信息
真: 0.384 (38.4%) ← 保留自己的信息

Step 4: 加权求和 V → 新向量

$\text{output} = 0.307 \times V_0 + 0.309 \times V_1 + 0.384 \times V_2$

维度0: $0.307 \times 1.3 + 0.309 \times 0.2 + 0.384 \times (-0.2) = 0.384$

维度1: $0.307 \times (-0.2) + 0.309 \times 1.0 + 0.384 \times 0.4 = 0.402$

维度2: $0.307 \times 0.4 + 0.309 \times 0.3 + 0.384 \times (-0.5) = 0.024$

$\text{output}_2 = [0.384, 0.402, 0.024]$

对比变化 — “真”的向量在 Attention 前后

Embedding 后: $[0.3, 0.4, -0.5]$

← 只知道“我是真”



Attention 后: $[0.384, 0.402, 0.024]$

← 融入了“天”和“气”的信息

最大变化

维度2从 -0.5 跳到 0.024 — 吸收了“天”“气”的正值。之后还经过残差连接 $x + \text{output}$ 和 MLP 进一步变换

因果遮罩 Causal Mask

GPT 是生成模型: 预测时只能看到前面的 token, 不能偷看后面

遮罩前(原始分数)

2.1	$-\infty$	$-\infty$	$-\infty$
1.1	1.8	$-\infty$	$-\infty$
0.9	1.2	2.0	$-\infty$
0.6	0.3	1.1	1.9



Softmax 后(注意力权重)

1.00	0.00	0.00	0.00
0.33	0.67	0.00	0.00
0.18	0.25	0.57	0.00
0.12	0.09	0.20	0.59

```
tril = torch.tril(torch.ones(T, T)) # 下三角矩阵
```

Self-Attention 全过程一页总结

以“天气真好”中“真”为例的完整 4 步计算

Step 1: 生成 Q/K/V

每个 token $\times W_q/W_k/W_v \rightarrow$ 三个视角向量

Step 2: $Q \cdot K^T$ 点积

$\text{score}(\text{真} \rightarrow \text{天}) = 0.10$ $\text{score}(\text{真} \rightarrow \text{气}) = 0.11$
 $\text{score}(\text{真} \rightarrow \text{真}) = 0.49$ $\text{score}(\text{真} \rightarrow \text{好}) = -\infty$

Step 3: Softmax

天: 0.307 (30.7%) 气: 0.309 (30.9%)
真: 0.384 (38.4%)

Step 4: 加权求和 V

$0.307 \times V_{\text{天}} + 0.309 \times V_{\text{气}} + 0.384 \times V_{\text{真}} \rightarrow$
 $[0.384, 0.402, 0.024]$

前后对比

Embedding 后

"真" = $[0.3, 0.4, -0.5]$ $\leftarrow$ 只知道"我是真"

Attention 后

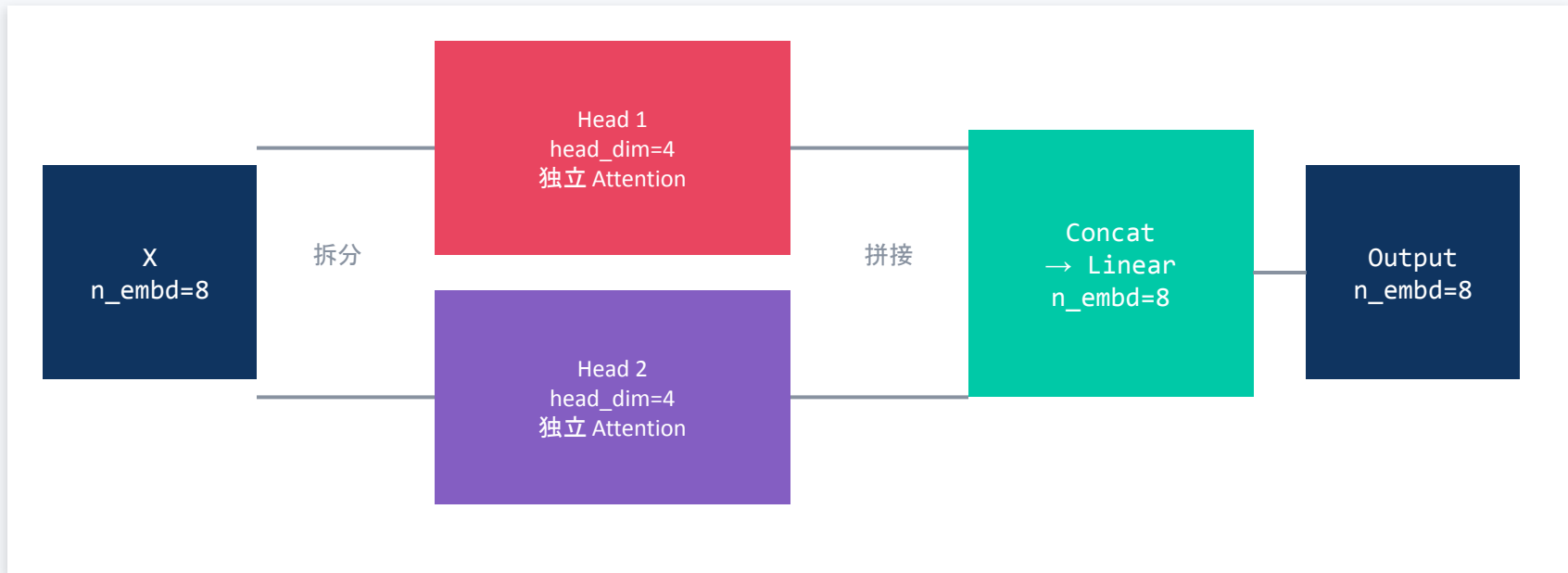
"真" = $[0.384, 0.402, 0.024]$ $\leftarrow$ 融入了上下文

核心机制

Attention 本质 = 每个 token 按相关度从其他 token 那里“借”信息, 改写自己的向量——这就是“融入上下文”

多头注意力 Multi-Head Attention

把 QKV 拆成多个"头", 每个头独立做 Attention, 最后拼接



$n_head=2$, $head_dim = n_embd/n_head = 8/2 = 4$. 多头 = 多视角, 不同头关注不同语义模式

单头 vs 多头 Attention

体现在 Self-Attention 这一步: Q/K/V 的生成和计算方式

单头 Attention

只有一组 W_q 、 W_k 、 W_v , 整个 C 维向量一起算一次注意力

```
x (B,T,C)
  ↓ × Wq, Wk, Wv
Q,K,V 各一组 (B,T,C)
  ↓ 一次点积
一组权重 → 一个输出
```

只从一个角度看相关性

多头 Attention (h 个头)

把 C 维拆成 h 份, 每份独立算一次 Attention, 最后拼回来

```
x (B,T,C)  C=6, h=2 为例
  ↓ 拆分: 每个头分到 C/h = 3 维
Head 0: [0.9, -0.2, 0.4] → Q0, K0, V0
Head 1: [0.7, 0.1, -0.3] → Q1, K1, V1
  ↓ 各自独立做 Attention
  ↓ concat(out0, out1) × Wo
最终输出 (B,T,C)
```

为什么要多头? ——每个头可以学到不同的关注模式

Head 0 可能学到关注语法关系
“真”关注“气”，因为是修饰词

Head 1 可能学到关注语义关系
“真”关注“天”，因为主题是天气

Head 2 可能学到关注位置关系
总是关注前一个或后一个 token

nanoGPT 实际实现

```
# Wq 还是一个大矩阵 (C, C)
q = (x @ Wq).view(B,T,n_head,C//n_head)
    .transpose(1,2)
# → (B, n_head, T, C//n_head)
```

关键理解

- 不是真的有 h 个小矩阵, 而是一个大矩阵等价完成
- 一次乘法算出所有头的 Q, 然后 reshape 拆开
- 参数量不变, 但学到的模式更丰富

残差连接 — 为什么 $x+f(x)$ 比 $f(x)$ 好？

信息保留的秘密：原始信号永远有一条直通路径

✗ 无残差 = 传话游戏

每一层完全重写信息，之前的细节可能丢失

无残差：信息逐层失真

```
x0 = embed("天气真好")    # 原始信号
x1 = Block1(x0)           # 可能丢失词义
x2 = Block2(x1)           # 可能丢失语序
x3 = Block3(x2)           # 可能丢失语气
```

✓ 有残差 = 草稿迭代

每一层只“添加注解”，原文始终保留

有残差：信息叠加保留

```
x0 = embed("天气真好")    # 原始信号
x1 = x0 + Block1(x0)    # x0 保留！
x2 = x1 + Block2(x1)    # x0+x1 都在
x3 = x2 + Block3(x2)    # 全部累加
```

梯度消失的数学解释

无残差： $\partial L / \partial x_0 = f'_3 \times f'_2 \times f'_1$ —— 连乘，每个 $f' < 1$ 就会指数级缩小 → 梯度消失

有残差： $\partial L / \partial x_0 = (1+f'_3)(1+f'_2)(1+f'_1)$ —— 展开后至少有一个“1”，梯度永远不为零

形象类比：无残差像“传话游戏”——每传一次就失真；有残差像“草稿批注”——原稿始终在手
“1”的存在 = 恒等映射 (identity shortcut)，让梯度可以“走捷径”直达输入层

✨ 核心洞察：残差 = “信息高速公路”，让梯度和信息都能跳过任意多层直达目的地

LayerNorm — 把分散的点拉回中心

每个 Token 独立归一化, 稳定训练、加速收敛

LayerNorm 计算过程(天 token)

输入: $x_0(\text{天}) = [3.2, -1.5, 0.8, 2.1]$ (C=4 示例)

Step 1: 计算均值

$$\mu = (3.2 + (-1.5) + 0.8 + 2.1) / 4 = 1.15$$

Step 2: 计算方差

$$\begin{aligned}\sigma^2 &= \text{mean}((3.2-1.15)^2, (-1.5-1.15)^2, \dots) \\ &= \text{mean}(4.20, 7.02, 0.12, 0.90) = 3.06\end{aligned}$$

Step 3: 归一化

$$\begin{aligned}\hat{x} &= (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} \\ &= [1.17, -1.51, -0.20, 0.54]\end{aligned}$$

Step 4: 仿射变换 (可学习参数)

$$y = \gamma \times \hat{x} + \beta \quad \# \gamma, \beta \text{ shape: } (C,)$$

为什么需要稳定?

- 激活值过大/过小 $\rightarrow$ 梯度爆炸/消失
- 不同维度尺度差异大 $\rightarrow$ 优化困难
- 深层网络累积效应 $\rightarrow$ 数值越来越极端
- LN 把每个 token 的 C 维拉回 $\mu=0, \sigma=1$

Pre-Norm vs Post-Norm

Post-Norm (原始 Transformer):

$$x = \text{LN}(x + \text{Attn}(x)) \text{ — 残差之后再归一}$$

Pre-Norm (GPT-2/nanoGPT):

$$x = x + \text{Attn}(\text{LN}(x)) \text{ — 归一之后再进 Block}$$

Pre-Norm 训练更稳定, 不需 warmup

✨ LN 公式: $y = \gamma \times (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$, 沿 C 维度计算, 每个 token 独立归一。参数量: $2C = 768$ (微乎其微)

MLP — 每个 Token 的独立思考器

先“扩张视野”，再“压缩决策”——非线性的关键



MLP forward() — nanoGPT

```
class MLP(nn.Module):
    def __init__(self, config):
        self.c_fc = nn.Linear(C, 4*C) # 384→1536
        self.gelu = nn.GELU()
        self.c_proj = nn.Linear(4*C, C) # 1536→384
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x): # x: (B,T,C)
        x = self.c_fc(x) # (B,T,4C)
        x = self.gelu(x) # 非线性激活
        x = self.c_proj(x) # (B,T,C)
        x = self.dropout(x)
        return x # 形状不变!
```

Attention 之后还需要 MLP?

- Attention = 线性加权求和, 只能“混合”信息
- MLP = 非线性变换, 能“提炼/创造”新特征
- 类比: Attn“开会听意见” + MLP“回去独立思考”
- GELU 提供平滑非线性: 小值软开关, 大值通过

参数量统计

c_fc: $C \times 4C = 384 \times 1536 = 589,824$
c_proj: $4C \times C = 1536 \times 384 = 589,824$
bias: $4C + C = 1920$

合计 $\approx 1.18M \times 4$ (bias) $\approx 4.72M$
占 Block 参数的 2/3 !

为什么 4C? 超参数, 实践发现 4x 是性能与成本的最佳平衡

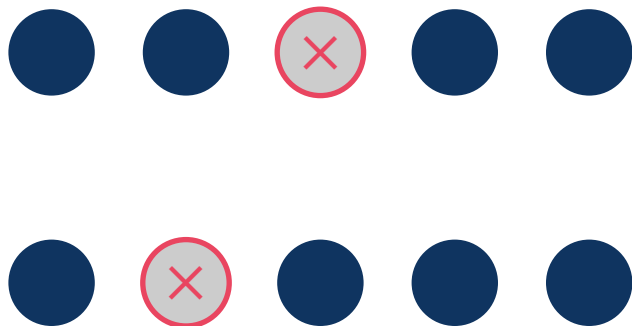
✨ MLP = “个人思考时间”: 每个 token 独立过 MLP, 不看其他 token。参数量占 2/3, 是模型知识的主要载体。

Dropout

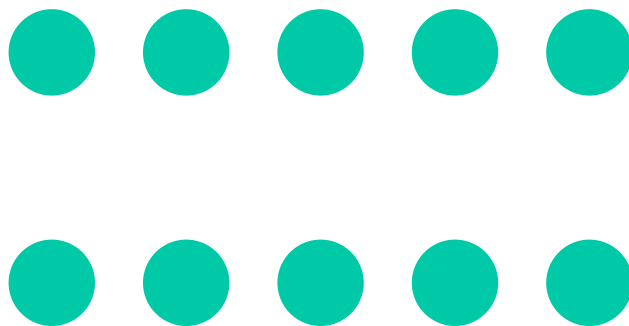


训练时随机"关闭"一部分神经元(设为 0), 推理时全部激活

训练时 ($p=0.2$)



推理时 (全部激活)



nanoGPT 中的位置: Attention 权重后 + MLP 输出后

```
self.dropout = nn.Dropout(dropout)
```

Softmax vs LayerNorm — 完全不同的工具

一个“分配概率”，一个“稳定数值”——别再混淆

Softmax

公式: $P(i) = \exp(x_i) / \sum \exp(x_j)$

特性:

- 输出 $\in (0,1)$, 总和=1(概率分布)
- 放大差异(大的更大, 小的更小)
- 不改变维度, 不学习参数

用途 1: Attention 内部

$\text{scores}(T,T) \rightarrow \text{softmax} \rightarrow$ 注意力权重
「真」看「天」0.45, 看「气」0.35, 看「真」0.20

用途 2: 最终输出

$\text{logits}(V) \rightarrow \text{softmax} \rightarrow$ 下一词概率
 $P(\text{"啊"})=0.35, P(\text{"呀"})=0.22, P(\text{"!"})=0.18$

LayerNorm

公式: $y = \gamma(x-\mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$

特性:

- 输出均值 ≈ 0 , 方差 ≈ 1 (标准化)
- 保持相对大小关系不变
- 有可学习参数 γ, β (共 $2C$ 个)

出现位置:

- Block 内 Attention 前 (Pre-Norm)
- Block 内 MLP 前 (Pre-Norm)
- Output Head 前 (Final LN)

操作维度: 沿 C 维度

每个 token 独立归一, 不看其他 token

✨ Softmax = “谁分多少蛋糕”(竞争关系), LayerNorm = “拉回同一起跑线”(独立操作)

Attention 横向交流 vs MLP 纵向深化

一个看“别人”，一个看“自己”

Attention — T 维度操作

- Token 之间交流, 每个 token 看所有其他 token
- 输出 = 其他 token 的加权和

Attention 核心计算

```
# Q, K, V: (B, nh, T, hs)
att = Q @ K.T / sqrt(hs)      # (T, T) 矩阵
att = softmax(att)            # 每行和=1
out = att @ V                  # 加权求和
# 「真」的输出 = 0.45×V(天) + 0.35×V(气) + 0.20
                             # ×V(真)
```

MLP — C 维度操作

- 每个 token 独立过 MLP, 不看其他 token
- 输出 = 非线性变换后的新表示

MLP 核心计算

```
# x: (B, T, C) — 每个 token 独立
x = linear_1(x)             # C → 4C 扩张
x = GELU(x)                 # 非线性激活
x = linear_2(x)             # 4C → C 压缩
# 「天」「气」「真」「好」各自过 MLP, 互不干扰
```

组件

Attention
MLP
LayerNorm
Residual

操作维度

T(横向)
C(纵向)
C(纵向)
全维度

作用

上下文感知
非线性变换
数值稳定
信息保留

类比

开会听意见
回去独立思考
拉回起跑线
备份原稿

💡 Attention 和 MLP 互补: 先“看周围”收集信息, 再“独立思考”深理解。两者缺一不可。

为什么需要 N 个 Block？

“多轮开会 + 思考”——每一层看得更深

Block 1-2

语法层

识别词类、词序、基本搭配

「苹果」= 名词, 「发布」= 动词

Block 3-4

消歧层

结合上下文确定词义

「苹果发布」→ 苹果=公司, 非水果

Block 5-8

语义层

理解意图、情感、事件关系

「发布新手机」→ 产品发布会场景

Block 9-12

生成策略层

决定如何继续、语气、风格

下一词可能: 「, 」「。」「并」

Depth vs Width vs MLP

- Depth (层数 N): 能看多深的抽象
- Width (头数 nh): 能同时看多少角度
- MLP (4C): 能存多少知识

三者互补, 不可替代:

浅而宽 ≠ 深而窄, 因为“抽象层次”需要深度

“多轮开会 → 思考”类比:

第1轮

开会: 收集信息

思考: 初步理解

第2轮

开会: 交叉验证

思考: 深入分析

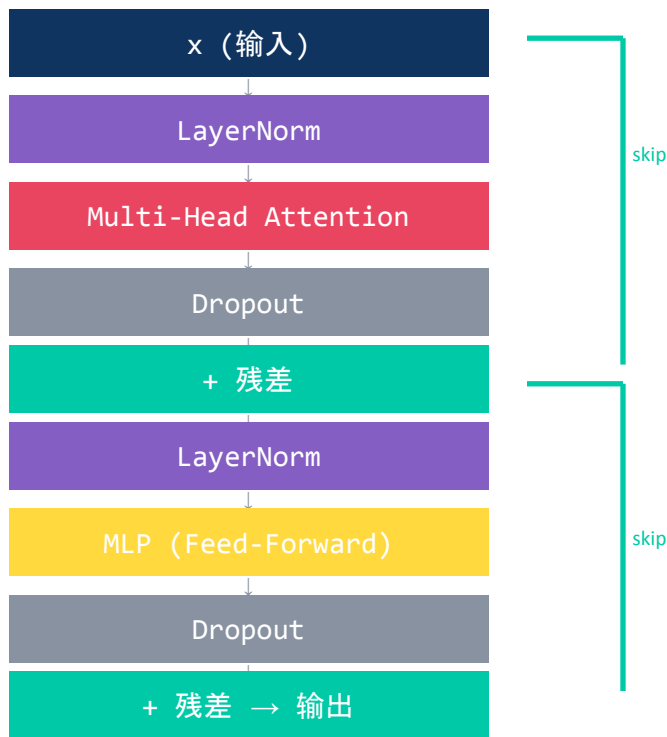
第3轮

开会: 达成共识

思考: 最终决策

🌟 N 个 Block = N 轮“开会+思考”, 每一层的理解都建立在前一层之上, 逐步从词法→语义→策略

Transformer Block 全景



一个 Block 的公式

```
# Attention 分支  
 $x = x + \text{attn}(\text{ln1}(x))$ 
```

```
# MLP 分支  
 $x = x + \text{mlp}(\text{ln2}(x))$ 
```

nanoGPT 堆叠

n_{layer} 个 Block
通常 6 ~ 12 层

每一层都在提炼更
高层次的语义特征

六大组件各司其职

Transformer Block 的每个部件都有明确分工

Attention

看周围 — 横向上下文感知

每个 token “开会”，看其他 token。操作 T 维度，输出 = 加权和。

MLP

想清楚 — 纵向非线性深化

每个 token “独立思考”。
 $C \rightarrow 4C \rightarrow \text{GELU} \rightarrow C$ ，占参数量 2/3。

残差连接

稳得住 — 恒等路径保护梯度

$x + f(x)$ 让梯度“走捷径”，解决深层网络梯度消失问题。

LayerNorm

拉回来 — 数值归一化

沿 C 维度归一为 $\mu=0, \sigma=1$ 。防止数值爆炸，稳定训练。

N个Block

看得深 — 逐层加深抽象

每层 = Attn+MLP。浅层看词法，深层看语义和策略。

Output Head

翻译回 — $C \rightarrow V$ 唯一映射

线性层 $W(C,V)$ 把隐藏向量映射回词表空间，生成 logits。

✨ 每个组件都不可替代: Attention “看”、MLP “想”、残差 “保”、LN “稳”、多层 “深”、Head “译”

Output Head — $C \rightarrow V$ 投影

`nn.Linear(C, V)` 把内部语义翻译回词表空间

矩阵乘法+偏置 — 以“好”的向量 $[0.6, 0.3, 0.9]$ 为例

```
lm_head = nn.Linear(3, 5)
```

```
logit(天) =  $0.6 \times 0.5 + 0.3 \times 0.2 + 0.9 \times 0.9 + 0.1 = 1.27$ 
```

```
logit(气) =  $0.6 \times (-0.3) + 0.3 \times 0.7 + 0.9 \times (-0.6) + (-0.1) = -0.61$ 
```

```
logit(真) =  $0.6 \times 1.2 + 0.3 \times (-0.5) + 0.9 \times 0.3 + 0.0 = 0.84$ 
```

```
logit(好) =  $0.6 \times 0.8 + 0.3 \times 0.1 + 0.9 \times 0.7 + 0.2 = 1.34 \leftarrow \text{最高}$ 
```

```
logit(冷) =  $0.6 \times (-0.4) + 0.3 \times 0.9 + 0.9 \times (-0.1) + (-0.2) = -0.26$ 
```

```
logits = [1.27, -0.61, 0.84, 1.34, -0.26]
```

天 气 真 好 冷

为什么需要 $C \rightarrow V$?

C 维 = 语义空间

模型内部的“思考语言”，连续浮点数，人类看不懂

V 维 = 词表空间

对应真实世界每个词，每一维=一个具体的词

类比：脑中想法(C维)

→ 嘴里说出具体的字(V维)

序列 0: “天气真好” → 预测下一个词

```
Softmax → [0.07, 0.03, 0.51, 0.28, 0.08]
```

天 气 真↑ 好 冷

预测：“真”概率最高 (51%)

序列 1: “天气真冷” → 预测下一个词

```
Softmax → [0.38, 0.10, 0.17, 0.07, 0.28]
```

天↑ 气 真 好 冷

预测：“天”概率最高 (38%)

结论

不同上下文导致不同预测——这就是语言模型理解上下文的核心机制

最后三步 — Final LN → Output Head → Softmax

从隐藏向量到词概率的最后一程

Step 3

Final LayerNorm

归一化最后一层的输出，确保数值稳定。每个 token 独立沿 C 维度归一。

$(B, T, C) \rightarrow (B, T, C)$
参数: $2C = 768$

Step 4

Output Head

线性层 $W(C,V)$ 把 384 维隐藏向量映射到 V 维词表空间，得到每个词的“分数”(logits)。

$(B, T, C) @ (C, V) \rightarrow (B, T, V)$
 $V = \text{词表大小}$

Step 5

Softmax

把 logits 转换为概率分布。exp 放大差异，归一化确保总和=1。

$\text{logits}(V) \rightarrow P(V)$
无参数，纯计算

「天气真好」最后三步实例

```
# Block×6 输出: x(好) = [0.82, -0.45, 1.23, ..., -0.67] shape: (1,4,384)
```

```
# Step 3: Final LayerNorm
```

```
x_ln = LayerNorm(x) # 每个 token 独立归一 → (1,4,384)
```

```
# x_ln(好) = [0.51, -0.89, 1.02, ..., -0.34] #  $\mu \approx 0, \sigma \approx 1$ 
```

```
# Step 4: Output Head
```

```
logits = x_ln @ W_out # (1,4,384) @ (384,V) → (1,4,V)
```

```
# logits(好)[3] = [2.1, -0.5, 0.8, ..., 3.4, 1.2] # V 个分数
```

✨ 最后三步的维度变化: $(B,T,C) \rightarrow \text{LN} \rightarrow (B,T,C) \rightarrow \text{Head} \rightarrow (B,T,V) \rightarrow \text{Softmax} \rightarrow \text{概率分布}$

```
# P("啊")=0.35, P("呀")=0.22, P("!")=0.18, P("天")=0.08 ...
```

Softmax 的两次出现

同一个函数, 两个完全不同的用途

① Attention 内部 — 分配关注度

将 QK^T scores 转为注意力权重

每行和=1, 表示“看各个 token 的比例”

「真」token 的注意力分配

```
# Q(真) @ K.T → raw scores:
scores = [3.2, 2.8, 1.5]    # 对[天,气,真]

# softmax:
P(天)=0.45, P(气)=0.35, P(真)=0.20

# 加权求和:
out = 0.45*V(天) + 0.35*V(气) + 0.20*V(真)
# 「真」最关注「天」, 其次「气」
```

操作维度: T (token间) | 输入输出: (T,T)

② 最终输出 — 生成词概率

将 logits 转为概率分布

总和=1, 表示“生成每个词的可能性”

位置 3(「好」之后)的输出

```
# Output Head 输出 logits:
logits = [2.1, -0.5, 0.8, ..., 3.4]    # V 个值

# softmax:
P("啊")=0.35, P("呀")=0.22
P("!")=0.18, P("天")=0.08 ...

# 从概率分布中采样:
```

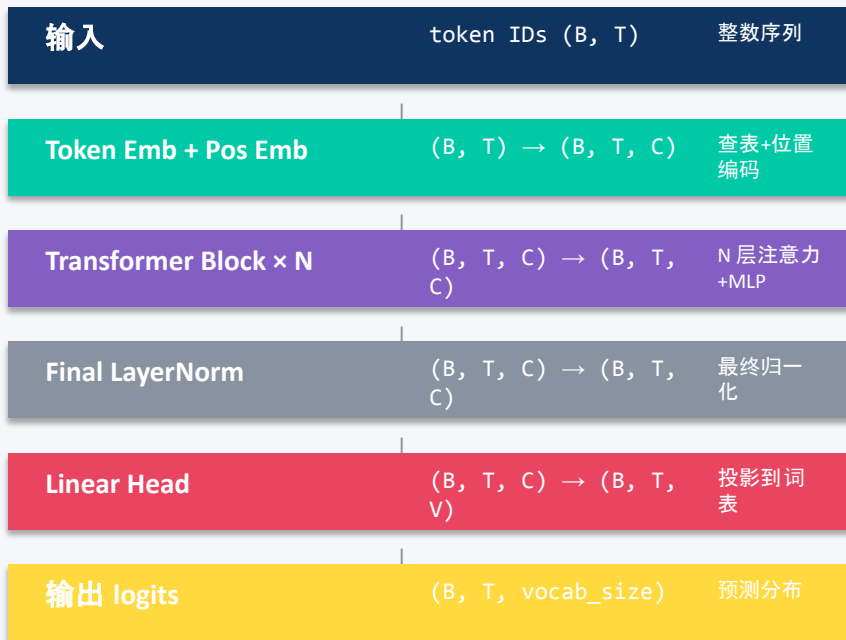
```
next_token = sample(probs)    # → "啊"
```

操作维度: V (词表) | 输入输出: (V,V)

✨ 两次 Softmax 的共同点: “总和=1 的竞争分配”。不同点: 一个分配“注意力”, 一个分配“词概率”

GPT 完整前向传播

从输入 token IDs 到输出 logits 的端到端数据流



nanoGPT forward 函数

GPT.forward(self, idx, targets=None)

```
# idx: (B, T) token IDs
B, T = idx.size()

# 1. Embedding
tok_emb = self.wte(idx)      # (B,T,C)
pos_emb = self.wpe(pos)      # (T,C)
x = tok_emb + pos_emb        # (B,T,C)

# 2. N 个 Transformer Block
for block in self.blocks:
    x = block(x)              # (B,T,C)

# 3. Final LayerNorm
x = self.ln_f(x)              # (B,T,C)

# 4. Linear Head → logits
logits = self.lm_head(x)      # (B,T,V)
```

架构 → 训练的桥梁

forward 产生 logits → 与 targets 算 cross_entropy loss → backward 更新参数 → 下一页详解

GPT 参数量推导

参数量逐项拆解

① Token Embedding $V \times C$

② Position Embedding $T \times C$

③ 每个 Transformer Block ($\times N$)

└ LayerNorm $\times 2$	$4C$	// γ, β 各 C
└ QKV 投影	$3C^2 + 3C$	// Q, K, V 投影
└ 输出投影	$C^2 + C$	// 注意力输出
└ MLP fc1	$4C^2 + 4C$	// 扩维 $\times 4$
└ MLP fc2	$4C^2 + C$	// 降回原维

单个 Block 合计 $\approx 12C^2 + 13C$

④ N 个 Block $N \times (12C^2 + 13C)$

⑤ 最终 LayerNorm $2C$

⑥ 输出 Head $C \times V$ (与①共享权重)

总参数量 $\approx V \cdot C + T \cdot C + N \cdot (12C^2 + 13C) + 2C$

核心直觉

• 参数量由 $N \times 12C^2$ 主导

参数量公式直觉

$N \times 12C^2$ 主导 · C 翻倍参数量翻 4 倍 · nanoGPT 10.8M $\rightarrow$ GPT-2 124M $\rightarrow$ GPT-3 175B

nanoGPT Small 数值计算

$V=65$ $C=384$ $N=6$ $T=256$ $h=6$

Token Emb	65×384	24,960
Position Emb	256×384	98,304
每个 Block	$12 \times 384^2 + 13 \times 384$	1,774,464
6 个 Block	$6 \times 1,774,464$	10,646,784
最终 LN		768

$\approx 10,770,816 \approx 10.8M$ 参数

规模对比

nanoGPT Small	10.8M	$C=384, N=6$
GPT-2 Small	124M	$C=768, N=12$
GPT-2 Large	774M	$C=1280, N=36$
GPT-3	175B	$C=12288, N=96$

nanoGPT Forward — 代码与概念对应

从代码到原理, 一一对应

nanoGPT GPT.forward()

```
def forward(self, idx, targets=None):
    B, T = idx.size()

    # Step 1: Embedding =====
    tok_emb = self.transformer.wte(idx)      # (B,T,C)
    pos_emb = self.transformer.wpe(pos)      # (T,C)
    x = self.transformer.drop(tok_emb + pos_emb)

    # Step 2: Block x N =====
    for block in self.transformer.h:
        x = block(x)                        # (B,T,C)
    # block(x): x = x + attn(ln1(x))
    #           x = x + mlp(ln2(x))

    # Step 3: Final LayerNorm =====
    x = self.transformer.ln_f(x)            # (B,T,C)

    # Step 4: Output Head =====
    logits = self.lm_head(x)                # (B,T,V)

    # Step 5: Loss (training only) =====
    if targets is not None:
        loss = F.cross_entropy(
            return logits, loss
```

Step 1: Embedding

wte: Token ID $\rightarrow$ 向量 (V,C)

wpe: 位置 $\rightarrow$ 向量 (T,C)

两者相加 $\rightarrow x_0$ (B,T,C)

Step 2: Block $\times$ N

每个 Block = LN $\rightarrow$ Attn $\rightarrow$ 残差 $\rightarrow$ LN $\rightarrow$ MLP $\rightarrow$ 残差
形状始终 (B,T,C), 内容逐层深化

Step 3-4: LN + Head

Final LN 稳定数值

lm_head: (B,T,C) @ (C,V) $\rightarrow$ logits

Step 5: Loss

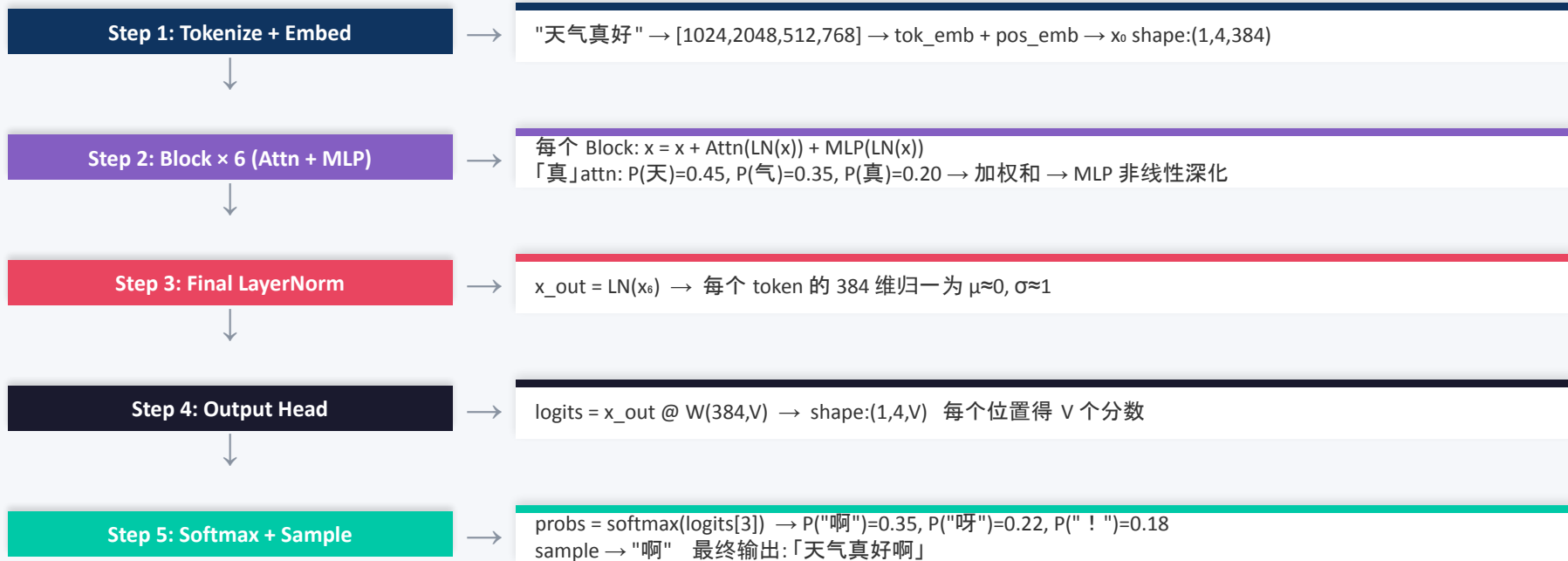
cross_entropy(logits, targets)

= $-\log P(\text{正确词})$ —— 让模型给正确词更高概率

🌟 nanoGPT 的 forward() 只有 ~15 行有效代码, 却实现了完整的 Transformer 前向传播。简洁即优雅。

天气真好 — 端到端完整走一遍

用实际数值跟踪每一步的变化



数据流 维度变化:

(1,4) → (1,4,384) → [Block×6 保持] → (1,4,384) → (1,4,V) → 概率分布 → 采样得下一个 token

✨ 核心洞察: 整个过程 = "查表 → 多轮开会 + 思考 → 稳定 → 翻译回词表 → 概率采样", 每步都有明确职责

三个关键问题总结

Embedding 权重 / Self-Attention 过程 / $C \rightarrow V$ 投影

① 语义身份证

`nn.Embedding(V,C)`

的权重矩阵 ($V \times C$)

- 用 token ID 当行号查表取出
- 初始随机, 训练后学到语义
- 相近词获得相近向量
- 占总参数量 $V \times C$

② 融入上下文

Self-Attention 四步

- $Q \cdot K^T \rightarrow$ 算注意力分数
- $\div \sqrt{d} \rightarrow$ 缩放
- Softmax $\rightarrow$ 归一化为权重
- 加权求和 $V \rightarrow$ 新向量

本质: 按相关度从其他
token 借信息, 改写自己

③ $C \rightarrow V$ 投影

`nn.Linear(C,V)`

矩阵乘法

- 从语义空间翻译回词表空间
- C维: 模型内部思考 语言
- V维: 对应真实世界每个词
- 让模型给出“下一个词”的答案

全景管线

$(B,T) \rightarrow \text{Embedding} \rightarrow (B,T,C) \rightarrow \text{Transformer} \times N \rightarrow (B,T,C) \rightarrow \text{Output Head} \rightarrow (B,T,V)$
编号 “我是什么” “结合语境我是什么” “下一个词是什么”

一句话

真实 GPT 只是把 3 换成 768/1024/12288, 把 5 换成 50257 — 数据流的逻辑完全一样

全景回顾 — 从编号到预测

GPT 前向传播的完整数据流 维度变化

输入 (B, T)

整数矩阵 · 每个数字是一个 token 编号 · $B=2, T=4 \rightarrow [[0,1,2,3],[0,1,2,4]]$

Embedding (B, T, C)

C 维度“凭空出现” · Token Emb + Position Emb · 编号 $\rightarrow$ 语义向量

Transformer $\times N$ (B, T, C)

维度不变但内容被彻底改写 · Self-Attention + MLP · 每个 token 融合上下文

Output Head (B, T, V)

$\text{nn.Linear}(C,V)$ 矩阵乘法 · C维 $\rightarrow$ V维 · 从语义空间翻译回词表空间

一句话

B 是并发批次始终不变 · T 是序列长度始终不变 · C 是模型的“思考宽度” · V 是最终的词表预测维度

Loss 构建



```
loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
```

logits reshape 过程

```
logits: (B, T, vocab_size)
      ↓ view(-1, vocab_size)
logits: (B*T, vocab_size)

targets: (B, T)
      ↓ view(-1)
targets: (B*T,)
```

训练循环

```
for step in range(max_iters):
    xb, yb = get_batch()
    logits, loss = model(xb, yb)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Forward
前向传播



Loss
计算损失



Backward
反向传播



Step
更新参数

预训练 Pre-training



在大规模无标注文本上，通过"预测下一个 token"来学习语言规律

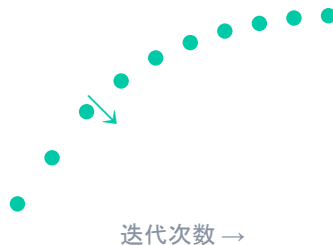
训练数据

- 维基百科
- 新闻文章
- 书籍 / 论文
- 代码 / 网页

关键超参数

```
learning_rate: 3e-4  
batch_size: 64  
warmup_steps: 200  
max_iters: 5000  
eval_interval: 500
```

Loss 曲线



预训练的目标: 让模型通过海量文本学会语言的统计规律、语法结构、世界知识。
预训练好的模型 = 通用语言能力基座，可以用于各种下游任务。

微调与 LoRA



全量微调 Full Fine-tuning



需要大量 GPU 显存

LoRA (低秩适配)



仅训练 $A \times B$, 参数量 $\ll W$

	全量微调	LoRA
训练参数	100%	0.1% ~ 1%
显存需求	极高	低
训练速度	慢	快
效果	最优	接近最优

推理与生成 Inference



自回归生成过程

Step 1: "今天天气"

Step 2: "今天天气真"

Step 3: "今天天气真好"

Step 4: "今天天气真好啊"

→ "真"

→ "好"

→ "啊"

→ "!"

循环直到 <EOS> 或达到最大长度

Temperature 温度

T=0.1 (低温 / 保守):

概率集中, 总是选最可能的词

T=1.0 (标准):

按原始概率采样

T=2.0 (高温 / 创意):

概率更平均, 更多随机性

Top-k & Top-p 采样

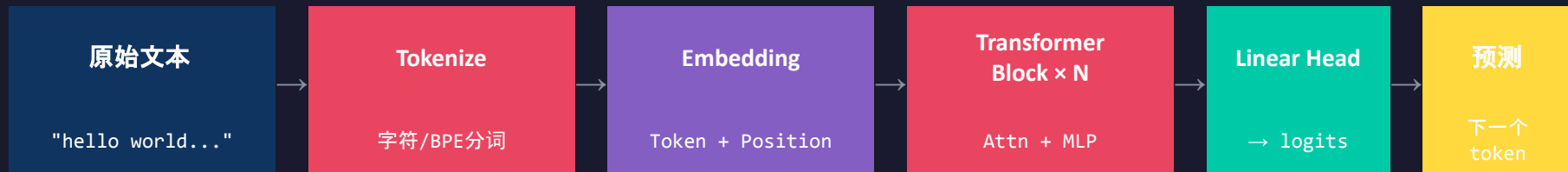
Top-k = 50:

只从概率最高的 50 个词中采样
过滤掉不合理的低概率词

Top-p = 0.9 (nucleus):

从累积概率达 90% 的词中采样
动态调整候选数量

总结:nanoGPT 端到端全景



模块一: Python 基础

数据结构 · 推导式
类与继承 · 装饰器



模块二: 数学基础

向量/矩阵 · 内积 · 转置
Softmax · 交叉熵 · 梯度



模块三: nanoGPT 核心

Tokenizer · Embedding · QKV
Attention · MLP · LayerNorm
预训练 · LoRA · 推理生成

理解 nanoGPT = 理解 GPT 的核心原理。掌握这些知识, 你就拥有了读懂任何 Transformer 模型的能力。